Proyecto-Funciones

- src
  - assets
  - components
  - .gitignore
  - app.here
  - index.js

```
1 function masterFunctions() {
2    return 'Código Limpio';
3 }
4
```

Funciones en JavaScript

El arte de estructurar código: De sintaxis básica a la maestría del 'Clean Code'.

Tema: Dark Mode.
Sintaxis: ES6+.
Listo para lectura.

NotebookLM

El Problema de la Repetición

Sin funciones, las acciones similares ensucian el código. Las funciones son los bloques de construcción que nos dan una única fuente de verdad.

```
// Login
alert('Hello everyone!');

// Logout
alert('Hello everyone!');
```

```
function showMessage() {
  alert('Hello everyone!');
}

showMessage(); // Login
showMessage(); // Logout
```

Evita la duplicación. Cambia el mensaje una vez, actualízalo en todas partes.

NotebookLM

Anatomía de una Declaración

Palabra clave.
Inicia la declaración.

Nombre de la función.
Debe ser un verbo descriptivo.

```
function showMessage(from, text) { /* body */ }
```

Parámetros.
Lista separada por comas (puede estar vacía).

Cuerpo.
El bloque de código que se ejecuta al llamar la función.

NotebookLM

El Alcance (Scope): Cajas de Cristal

Una función puede ver hacia afuera, pero el exterior no puede ver hacia adentro. Las variables locales viven y mueren dentro de la función.

```
let userName = 'John';                    // Variable externa (Global)

function showMessage() {                 // Variable local
  let message = 'Hello, I am JavaScript!'; // Variable local
                                         // Puede leer 'userName'
  alert( 'Hello, ' + userName );         //✕ ¡Error! Inaccesible desde afuera
}

alert( message );
}
```

NotebookLM

Shadowing (Sombreado de Variables)

Si una variable local tiene el mismo nombre que una externa, la local 'sombrea' (ignora) a la externa.

```
1  let userName = 'John';
2
3  function showMessage() {
4    let userName = 'Bob'; // Declara variable local
5    alert('Hello, ' + userName); // Usa 'Bob'
6  }
7
8  showMessage();
9
10 alert( userName ); // 'John' (Intacta)
```

NotebookLM

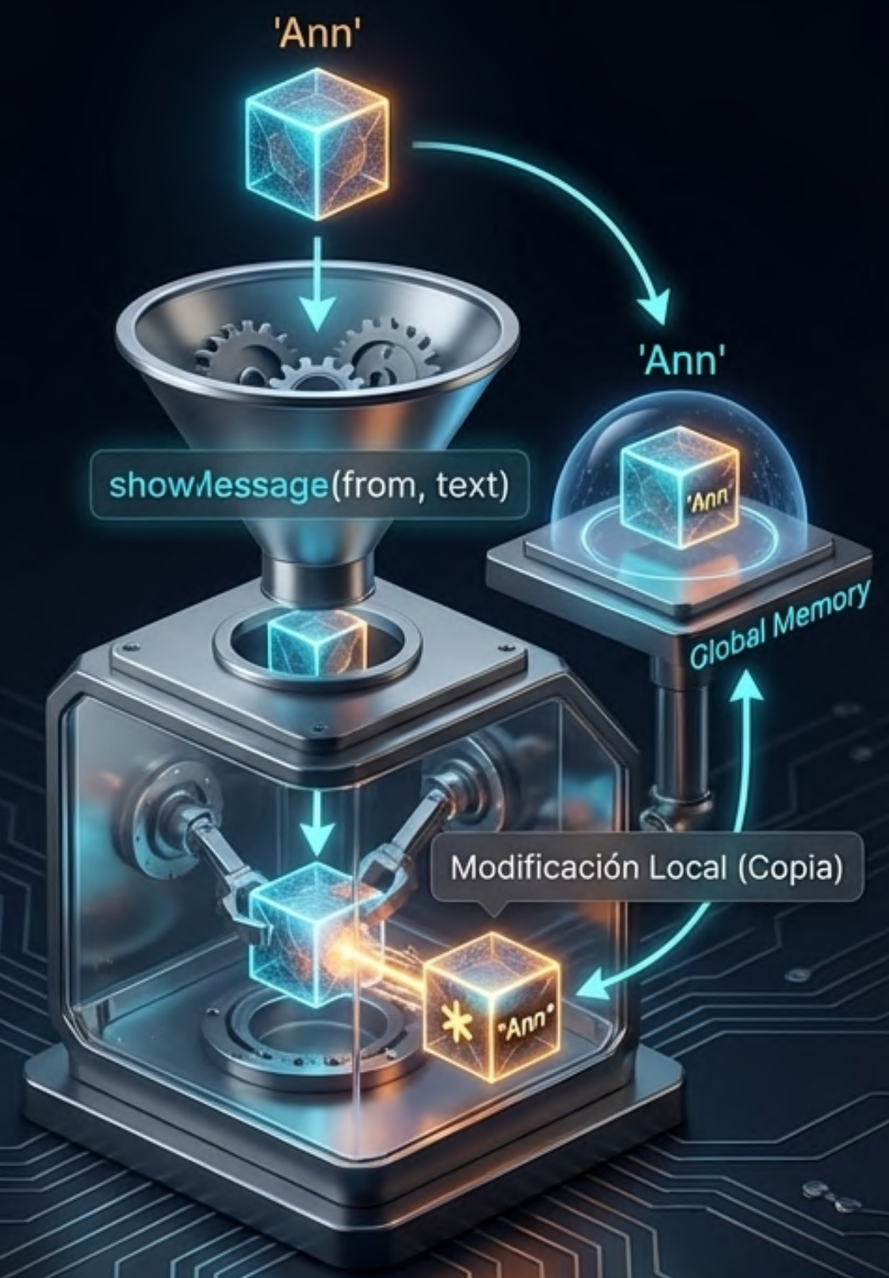
Parámetros vs. Argumentos

Parámetro:
Variable en la declaración (embudo de entrada).

Argumento:
El valor real pasado al llamar (el material).

```
function showMessage(from, text) {
  from = '*' + from + '*'; // Modifica una COPIA local
  alert( from + ': ' + text );
}

let from = 'Ann';
showMessage(from, 'Hello');
alert(from); // Ann (Original no cambia)
```



NotebookLM

La Evolución de los Valores por Defecto

Cuando se omite un argumento, su valor es undefined. Así es como JavaScript ha manejado esto a través del tiempo.

Era 1: El Control Manual (Antiguo)

```
if (text === undefined) {
  text = 'empty';
}
```

Era 2: El Operador Lógico (Clásico)

```
text = text || 'empty';
```

Era 3: Sintaxis Declarativa (Moderno/ES6)

```
function showMessage(from,
  text = 'no text given')
```

El valor por defecto (`text = anotherFunction()`) solo se ejecuta si el argumento falta o es estrictamente `undefined`.

NotebookLM

Matriz de Operadores: || vs ??

El operador `||` reacciona a cualquier valor `'falsy'`. El operador moderno `??` es más seguro porque solo reacciona a `null` o `undefined`.

Valor de Entrada	`|| 'default'`	`?? 'default'`
`undefined`	`'default'`	`'default'`
`null`	`'default'`	`'default'`
`''` (string vacío)	`'default'`	`''` (✅ Protegido)
`0` (cero)	`'default'`	`0` (✅ Protegido)

```
function showCount(count) {
  alert(count ?? 'unknown');
}

showCount(0); // Muestra 0, no 'unknown'
```

NotebookLM

El Retorno (Extrayendo el Resultado)

La directiva `return` detiene la ejecución inmediatamente y devuelve el valor al código que hizo la llamada.

```
function sum(a, b) {
  return a + b; // La máquina escupe el resultado
}

let result = sum(1, 2); // Capturamos el producto
alert(result ); // 3
```

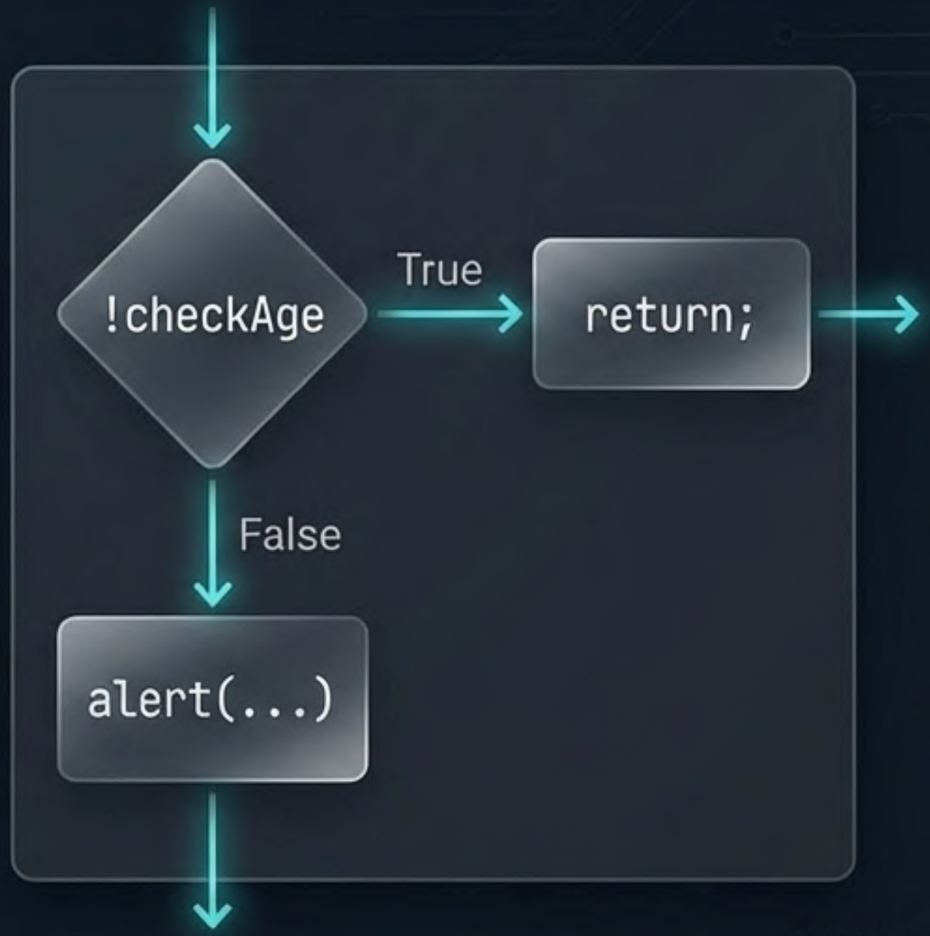
⚠ **Linter Note:** Una función sin return, o con un return; vacío, siempre devuelve undefined.

NotebookLM

Retornos Tempranos (Early Exit)

Usar return vacío funciona como una compuerta de seguridad. Evita anidar lógica profundamente con sentencias else.

```
function showMovie(age) {
  if ( !checkAge(age) ) {
    return; // Salida temprana: no avanza al alert
  }
  alert( 'Mostrando la película...' );
}
```



NotebookLM

La Trampa del Retorno

Nunca pongas un salto de línea entre return y su valor. JavaScript asume un punto y coma automático (ASI), matando tu función.

❌ Antipatrón

```
return
  (some + long + expression);

// JS lo lee como: return;
```

✅ Patrón Correcto

```
return (
  some + long + expression
);
```

NotebookLM

Nomenclatura: El Vocabulario de tu Código

Las funciones son acciones. Su nombre suele ser un verbo breve y preciso. El equipo debe acordar el significado de cada prefijo.

`get...`

Retorna un valor.

Ej. `getAge()`

`calc...`

Realiza cálculos matemáticos y retorna.

Ej. `calcSum()`

`create...`

Instancia y retorna un nuevo objeto/elemento.

Ej. `createForm()`

`check...`

Evalúa una condición y retorna un Booleano.

Ej. `checkPermission()`

NotebookLM

Una Función = Una Acción

Una función debe hacer exactamente lo que su nombre sugiere, y nada más. Romper esta regla crea efectos secundarios inesperados.

Code Review: getAge()

```
function getAge() {
  alert(age);
}
```

✕ Mal. Muestra un alert en lugar de solo obtener/retornar el valor.

Code Review: createForm()

```
function createForm() {
  document.body.append(form);
}
```

✕ Mal. Modifica el DOM (efecto secundario) en lugar de solo crearlo.

Code Review: checkPermission()

```
function checkPermission() {
  alert('Access Denied');
}
```

✕ Mal. Debe retorm/false, no manejar la UI.

Code Review: checkPermission()

```
function checkPermission() {
  alert('Access Denied');
}
```

✕ Mal. Debe retornar true/false, no manejar la UI.

NotebookLM

Funciones == Comentarios

La existencia de una función bien nombrada es el mejor comentario.
El código se vuelve “autodescriptivo” y más fácil de probar.

Difícil de leer

```
function showPrimes(n) {
  nextPrime: for (let i = 2; i < n; i++) {
    for (let j = 2; j < i; j++) {
      if (i % j == 0) continue nextPrime;
    }
    alert( i );
  }
}
```

Código Limpio

```
function showPrimes(n) {
  for (let i = 2; i < n; i++) {
    if (!isPrime(i)) continue;
    alert(i);
  }
}

function isPrime(n) { ... }
```

NotebookLM

El Camino a la Maestría

Dominar las funciones, los retornos y el scope nos permite destilar la lógica compleja en expresiones de pura elegancia.

1. Básico (Estructura de control tradicional)

```
function checkAge(age) {
  if (age > 18) return true;
  return confirm('¿Permiso?');
}
```

2. Intermedio (Operador Ternario)

```
function checkAge(age) {
  return (age > 18) ? true : confirm('¿Permiso?');
}
```

3. Avanzado (Evaluación de Cortocircuito)

```
function checkAge(age) {
  return (age > 18) || confirm('¿Permiso?');
}
```

El código es máquina. El 'Clean Code' es lenguaje humano.
Las funciones son el vocabulario con el que escribes tu historia.

NotebookLM